

Nim

Язык программирования Введение и основы

От первой команды до замыканий и итераторов

Житков Никита · Сагун Александр · Жуков Александр

Данные и переменные / Функции / Управление потоком

История и спецификация

2005 Начало разработки. Автор — Андреас Румпф.

2008 Первая публичная версия Nimrod.

2009 Разработка компилятора полностью на Nimrod.

2014 Переименование языка в Nim.

2.2.12 Стабильная версия на 28.09.2026. Релиз — 08.09.2026.
Nim Manual — официальное руководство по языку.

Что отличает Nim

Компиляция

C / C++ для нативной сборки. JavaScript для JS-среды.

Статическая типизация

Типы проверяются до запуска, многие типы выводятся автоматически.

Выразительный синтаксис

Отступы, процедуры, объекты, обобщения, шаблоны и макросы.

Управление памятью

ORC по умолчанию в Nim 2.x. Подсчёт ссылок и сборка циклов.

Нативный код не требует VM Nim. Скорость зависит от алгоритма и параметров сборки.

Три способа запуска

Локально

ТЕРМИНАЛ В КАТАЛОГЕ src

```
nim c -r main.nim  
nim c -d:release -r main.nim
```

Docker

ПРИМЕР СБОРКИ И ЗАПУСКА

```
docker build -t nim-lessons .  
docker run --rm nim-lessons
```

МИНИМАЛЬНЫЙ DOCKERFILE

```
FROM nimlang/nim:latest  
WORKDIR /app  
COPY . .  
RUN nim c -d:release main.nim  
CMD ["/main"]
```

Онлайн

play.nim-lang.org

Установка: nim-lang.org/install.html

Для повторяемой сборки фиксируют версию образа.

Типы данных и переменные

p01_types.nim · ФРАГМЕНТ

```
var x: int = 42
var y = 3.14
var s = "Nim"
var b = true
var c = 'N'
var t = (1, "two", 3.0)
var arr: array[3, int] = [1, 2, 3]
var sq = @[1, 2, 3]
```

ВЫВОД ПРОЕКТА

```
int:    42
float:  3.14
string: Nim
bool:   true
char:   N
tuple:  (1, "two", 3.0)
array:  [1, 2, 3]
seq:    @[1, 2, 3]
```

var изменяемая переменная

let однократное присваивание

const значение времени компиляции

string и tuple могут изменяться через var.

array — фиксированный размер, seq — динамический.

char — байт; UTF-8 у string — соглашение.

Операции над типами

p02_operations.nim · ОПЕРАЦИИ

```
import std/strutils
let a = 10
let b = 3
echo a + b, " ", a - b, " ", a * b
echo a / b
echo a div b, " ", a mod b
echo "Hello, " & "World"
echo "Hello " & repeat("!", 3)
echo true and false
echo true or false
echo not true
echo 10 > 5, " ", 10 == 10
```

РЕЗУЛЬТАТЫ ИЗ ЛОГА

```
+ / - / * : 13 / 7 / 30
/      : 3.3333333333333335
div : 3
mod : 1
Hello, World
Hello !!!
false / true / false
true / true
```

/ даёт вещественный результат, div — целое частное, mod — остаток.

Преобразования типов

p03_typecast.nim · ПРЕОБРАЗОВАНИЯ

```
import std/strutils

echo int(3.99)
echo float(7)
echo $42
echo parseInt("123")
echo parseFloat("3.14")
```

ВЫВОД ПРОЕКТА

```
3
7.0
42
123
3.14
```

Для переменных `int` и `float` нужно явное преобразование.
Целочисленные литералы могут подходить к `float` по контексту.

`parseInt` и `parseFloat` при неверной строке вызывают `ValueError`.
`cast` переинтерпретирует представление, это не обычное преобразование.

Области видимости

p04_scope.nim · ФРАГМЕНТ

```
var globalVar = "я глобальная переменная"

block:
  var blockVar = "я в блоке"
  echo blockVar
# echo blockVar  # вне блока недоступна

proc inner() =
  echo globalVar

inner()
```

block создаёт
вложенную область.

Процедура видит
внешние объявления.

Глобальное имя
относится к модулю.
Для других модулей
нужны экспорт и импорт.

Вложенное объявление может затенить внешнее имя. Используйте различные имена.

Владение: sink и явный move

ПОВЕДЕНИЕ ПРИ ПОСЛЕДУЮЩЕМ ЧТЕНИИ

```
proc takeOwnership(s: sink string) =  
  echo "Получено: ", s  
  
var myStr = "hello"  
takeOwnership(myStr)  
echo myStr # hello
```

myStr используется после вызова: в sink передаётся копия.
Поэтому результат hello в логе проекта ожидаем.

ОТДЕЛЬНЫЙ ПРИМЕР ЯВНОГО ПЕРЕМЕЩЕНИЯ

```
takeOwnership(move(myStr))  
echo myStr.len # 0
```

move сбрасывает исходную строку в пустое состояние.

Процедуры: вход и выход

p06_functions.nim · ФРАГМЕНТ

```
proc greet(name: string = "world"): string =  
  result = "Hello, " & name & "!"
```

```
proc add(a, b: int): int = a + b  
proc add(a, b: float): float = a + b
```

```
echo greet()  
echo greet("Nim")  
echo add(2, 3)  
echo add(2.5, 3.5)
```

ВЫВОД ПРОЕКТА

Hello, world!

Hello, Nim!

5

6.0

Возврат: result, return
или последнее выражение.
Несколько значений — tuple.

proc допускает побочные эффекты. func объявляет процедуру без побочных эффектов.

Рекурсия

```
p07_recursion.nim · n >= 0

proc factorial(n: int): int =
  if n <= 1: 1
  else: n * factorial(n - 1)

proc fib(n: int): int =
  if n < 2: n
  else: fib(n - 1) + fib(n - 2)

echo factorial(5)
echo fib(10)
```

ВЫВОД ПРОЕКТА

5! = 120

fib(10) = 55

Базовый случай
останавливает рекурсию.

fib повторяет вычисления.
Для больших n нужны
цикл или мемоизация.

Оптимизация хвостовых вызовов не гарантируется. Глубина рекурсии ограничена стеком.

Замыкания и функции высшего порядка

p08_closures.nim · ФРАГМЕНТ

```
proc apply(f: proc(x: int): int; x: int): int =  
  f(x)  
  
proc makeCounter(): proc(): int =  
  var count = 0  
  result = proc(): int =  
    count.inc  
    count  
  
let double = proc(x: int): int = x * 2  
echo apply(double, 21)  
let counter = makeCounter()  
echo counter(), " ", counter(), " ", counter()
```

РЕЗУЛЬТАТЫ

42

1 2 3

apply принимает
процедуру.

counter сохраняет
доступ к count
между вызовами.

Условия и циклы

p09_control_flow.nim · ФРАГМЕНТ

```
for i in 0..4:
  if i mod 2 == 0:
    echo i, " - чётное"
  else:
    echo i, " - нечётное"

var n = 5
while n > 0:
  echo "n = ", n
  dec n

case n
of 0: echo "ноль"
of 1..5: echo "от 1 до 5"
else: echo "другое"
```

ВЫВОД ПРОЕКТА

```
0 - чётное
1 - нечётное
2 - чётное
3 - нечётное
4 - чётное

n = 5
n = 4
n = 3
n = 2
n = 1
ноль
```

when и именованный блок

УСЛОВИЕ ВРЕМЕНИ КОМПИЛЯЦИИ

```
when defined(release):  
    echo "release"  
else:  
    echo "debug"
```

when выбирает ветку
при компиляции.

`nim c -d:release -r main.nim`

В этом примере напечатает release.

ВЫХОД ИЗ ВЛОЖЕННЫХ ЦИКЛОВ

```
block outer:  
    for i in 0..10:  
        for j in 0..10:  
            if i * j == 20:  
                echo i, " ", j  
                break outer
```

break outer завершает
весь именованный блок.

Первая подходящая пара: 2 10.

Итераторы и yield

p10_iterators.nim · ФРАГМЕНТ

```
iterator countdown(n: int): int =  
  var i = n  
  while i >= 0:  
    yield i  
    dec i  
  
for x in countdown(3):  
  echo x
```

ВЫВОД ПРОЕКТА

3
2
1
0

yield передаёт очередное значение циклу for,
после чего итератор продолжает выполнение.

Что мы разобрали

01

Данные

Типы, переменные, операции и преобразования

02

Функции

Области видимости, sink, рекурсия и замыкания

03

Управление потоком

Условия, циклы, блоки и итераторы

Далее: исключения, модули и пакеты, обобщённые типы, коллекции, работа с файлами и консолью.

Официальные источники

Nim Manual	nim-lang.org/docs/manual.html
Nim Tutorial I и II	nim-lang.org/docs/tut1.html / tut2.html
Владение и перемещение	nim-lang.org/docs/destructors.html
Управление памятью	nim-lang.org/docs/mm.html
Операции со строками	nim-lang.org/docs/strutils.html
Установка и запуск онлайн	nim-lang.org/install.html / play.nim-lang.org
Релиз Nim 2.2.12	nim-lang.org/blog/2026/09/08/nim-2212.html

История: официальные релизы 0.8.6 (2009) и 0.10.2 (2014).